

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский университет ИТМО»
Факультет Программной инженерии и компьютерной техники

Лабораторная работа №2
по дисциплине «Основы программной инженерии»

Вариант: 3260

Преподаватель:
Егошин Алексей Васильевич

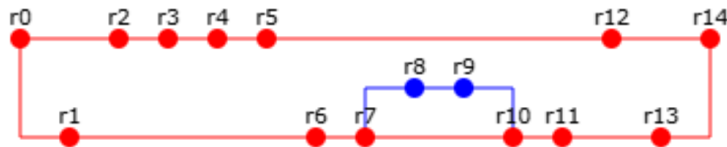
Выполнила:
Матвеева Полина Павловна
Группа: Р3215

Санкт-Петербург, 2026

Задание

Лабораторная работа #2

Вариант



Сконфигурировать в своём домашнем каталоге репозитории svn и git и загрузить в них начальную ревизию файлов с исходными кодами (в соответствии с выданным вариантом).

Воспроизвести последовательность команд для систем контроля версий svn и git, осуществляющих операции над исходным кодом, приведённые на блок-схеме.

При составлении последовательности команд необходимо учитывать следующие условия:

- Цвет элементов схемы указывает на пользователя, совершившего действие (красный - первый, синий - второй).
- Цифры над узлами - номер ревизии. Ревизии создаются последовательно.
- Необходимо разрешать конфликты между версиями, если они возникают.

Ход работы

Git

1 шаг - git init

КОММИТ	Команды
r0	<pre>git config user.name "red" git config user.email "red@gmail.com" git checkout -b branch-red1 #меняем файлы git add . git commit -m "Initial commit r0"</pre>
r1	<pre>git checkout -b branch-red2 #меняем файлы git add . git commit -m "commit r1"</pre>
r2	<pre>git checkout branch-red1 #меняем файлы git add . git commit -m "commit r2"</pre>
r3	<pre>#меняем файлы git add . git commit -m "commit r3"</pre>

r4	#меняем файлы git add . git commit -m "commit r4"
r5	#меняем файлы git add . git commit -m "commit r5"
r6	git checkout branch-red2 #меняем файлы git add . git commit -m "commit r6"
r7	#меняем файлы git add . git commit -m "commit r7"
r8	git checkout -b branch-blue #меняем файлы git add . git commit --author="blue <blue@gmail.com>" -m "commit r8"
r9	#меняем файлы git add . git commit --author="blue <blue@gmail.com>" -m "commit r9"
r10	git checkout branch-red2 git merge --no-commit branch-blue #исправляем конфликт ручками git add . #меняем файлы git add . git commit -m "commit r10"
r11	#меняем файлы git add . git commit -m "commit r11"
r12	git checkout branch-red1 #меняем файлы git add . git commit -m "commit r12"
r13	git checkout branch-red2 #меняем файлы git add . git commit -m "commit r13"
r14	git checkout branch-red1 git merge --no-commit branch-red2 #ручками исправляем git add . #меняем файлы git add . git commit -m "commit r14"

Скрипт для git

```
#!/bin/bash
```

```
set -x
```

```
#очищаем данные с прошлого запуска
```

```
rm -rf .git
rm -f .gitignore
```

```
find src -type f ! -name '.keep' -delete
```

```
#создание репозитория
git init
```

```
#настройка первого пользователя
git config user.name "red"
git config user.email "red@gmail.com"
```

```
git checkout -b branch-red1
```

```
#.gitignore
echo "commits" > .gitignore
echo ".backup" >> .gitignore
echo ".backupignore" >> .gitignore
echo "script.txt" >> .gitignore
echo "src/.keep" >> .gitignore
echo ".idea" >> .gitignore
echo ".gradle" >> .gitignore
echo "/build" >> .gitignore
echo "gradle*" >> .gitignore
```

```
#Initial commit
unzip -o commits/commit0.zip -d src
git add .
git commit -m "Initial commit r0"
```

```
#r1
git checkout -b branch-red2
unzip -o commits/commit1.zip -d src
git add .
git commit -m "commit r1"
```

```
#r2-r5
git checkout branch-red1
unzip -o commits/commit2.zip -d src
git add .
git commit -m "commit r2"
```

```
unzip -o commits/commit3.zip -d src
```

```
git add .  
git commit -m "commit r3"
```

```
unzip -o commits/commit4.zip -d src  
git add .  
git commit -m "commit r4"
```

```
unzip -o commits/commit5.zip -d src  
git add .  
git commit -m "commit r5"
```

```
#r6-r7  
git checkout branch-red2  
unzip -o commits/commit6.zip -d src  
git add .  
git commit -m "commit r6"
```

```
unzip -o commits/commit7.zip -d src  
git add .  
git commit -m "commit r7"
```

```
#r8-r9 blue user  
git checkout -b branch-blue  
unzip -o commits/commit8.zip -d src  
git add .  
git commit --author="blue <blue@gmail.com>" -m "commit r8"
```

```
unzip -o commits/commit9.zip -d src  
git add .  
git commit --author="blue <blue@gmail.com>" -m "commit r9"
```

```
#для r10 родители r7 и r9 сливаем их  
git checkout branch-red2
```

```
#исправляем конфликт ручками  
#git merge --no-commit branch-blue
```

```
git merge --no-commit branch-blue -Xtheirs  
git add .
```

```
unzip -o commits/commit10.zip -d src  
git add .  
git commit -m "commit r10"
```

```
#r11
unzip -o commits/commit11.zip -d src
git add .
git commit -m "commit r11"
```

```
#r12
git checkout branch-red1
unzip -o commits/commit12.zip -d src
git add .
git commit -m "commit r12"
```

```
#r13
git checkout branch-red2
unzip -o commits/commit13.zip -d src
git add .
git commit -m "commit r13"
```

```
#r14 - делаем merge r13 and r12
git checkout branch-red1
```

```
#ручками
#git merge --no-commit branch-red2
git merge --no-commit branch-red2 -Xours
git add .
```

```
unzip -o commits/commit14.zip -d src
git add .
git commit -m "commit r14"
```

```
#выводим граф
git log --graph --oneline --all
```

Ручное разбирание конфликтов происходит в vim

```

<<<<<<< HEAD
    public int[] ii() {
        return new int[]{0, 1, 2, 3, 4};
=====
    public byte oo() {
        return 2;
    }

    public String nn() {
        "".">+.+++++++..+++.>+.<+++++++++++.>.+...;
>>>>>>> branch-red2
    }
}

```

```

<<<<<<< HEAD
    public byte oo() {
        return 2;
=====
    public int[] ii() {
        return new int[]{4, 3, 2, 1};
    }

    public String nn() {
        return "+++++++[>++++++>+++++++>+++>+<<<<-]>+";
>>>>>>> branch-red2
    }
}

```

51,1

Svn

Коммит	Команды
Подготовка	<pre> #создание репозитория SVN_PATH="\$(pwd)/svn_repo" svnadmin create "\$SVN_PATH" #фрмируем URL REPO_URL="file://\$SVN_PATH" #создаем структуру svn mkdir -m "Create structure" "\$REPO_URL/trunk" "\$REPO_URL/branches" #checkout в рабочую копию svn checkout "\$REPO_URL" "working_copy" cd "working_copy" </pre>

r0	<pre>mkdir -p "trunk/src" unzip -o "../commits/commit0.zip" -d "trunk/src" svn add "trunk" svn commit -m "Initial commit r0" --username "red"</pre>
r1	<pre>svn copy "\$REPO_URL/trunk" "\$REPO_URL/branches/red2" -m "Create branch red2" --username "red" svn update #ветка red2 unzip -o "../commits/commit1.zip" -d "branches/red2/src" svn commit -m "commit r1" --username "red"</pre>
r2-r5	<pre>for i in {2..5} do unzip -o "../commits/commit\$i.zip" -d "trunk/src" svn commit -m "commit r\$i" --username "red" done</pre>
r6-r7	<pre>for i in {6..7} do unzip -o "../commits/commit\$i.zip" -d "branches/red2/src" svn commit -m "commit r\$i" --username "red" done</pre>
r8-r9(blue)	<pre>svn copy "\$REPO_URL/branches/red2" "\$REPO_URL/branches/blue" -m "Create branch blue" --username "blue" svn update for i in {8..9} do unzip -o "../commits/commit\$i.zip" -d "branches/blue/src" svn commit -m "commit r\$i" --username "blue" done</pre>
r10	<pre>#мердж blue в red2 cd "branches/red2" svn update svn merge "\$REPO_URL/branches/blue" --accept "theirs-full" unzip -o "../../commits/commit10.zip" -d "src" svn commit -m "commit r10" --username "red" cd "../.."</pre>
r11	<pre>unzip -o "../commits/commit11.zip" -d "branches/red2/src" svn commit -m "commit r11" --username "red"</pre>
r12	<pre>unzip -o "../commits/commit12.zip" -d "trunk/src" svn commit -m "commit r12" --username "red"</pre>
r13	<pre>unzip -o "../commits/commit13.zip" -d "branches/red2/src" svn commit -m "commit r13" --username "red"</pre>
r14	<pre>cd "trunk"</pre>

	<pre> svn update svn merge "\$REPO_URL/branches/red2" --accept "mine-full" unzip -o "../././commits/commit14.zip" -d "src" svn commit -m "commit r14" --username "red" cd ".." </pre>
--	---

Разрешение конфликтов в svn автоматически в nano

```

admin@DESKTOP-S4DVMR:/mnt/c/Users/admin/OneDrive - ITMO UNIVERSITY/Рабочий стол/
itmo/opn/lab2-opi/working_copy/trunk$ svn update
svn merge "$REPO_URL/branches/red2"
Updating '.':
At revision 17.
--- Merging r3 through r17 into '.':
U   src/A.java
C   src/C.java
C   src/G.java
C   src/I.java
U   .
--- Recording mergeinfo for merge of r3 through r17 into '.':
G   .
Summary of conflicts:
Text conflicts: 3
Merge conflict discovered in file 'src/C.java'.
Select: (p) Postpone, (df) Show diff, (e) Edit file, (m) Merge,
        (s) Show all options: df
--- src/C.java.working - MINE
+++ src/C.java - MERGED
@@ -1,4 +1,4 @@
-public class C implements A {
+public class C extends null implements A {

    private int g = 42;

@@ -19,6 +19,7 @@
    public void bb() {
        System.out.println(42);
    }
+<<<<<<< .working

    public java.lang.Class qq() {
        return getClass();
@@ -39,4 +40,31 @@
    public long ac() {
        return 111;
    }
+||||||| .merge-left.r2
+=====
+
+    public java.lang.Class qq() {
+        return getClass();
+    }

```

Выводы:

В процессе выполнения работы я на практике сопоставила распределённую архитектуру git с централизованным подходом svn, что позволило наглядно увидеть преимущества современных инструментов. Если в git работа с ветками реализована через удобные и легковесные указатели, обеспечивающие высокую гибкость и скорость, то в svn создание

ответвлкний путём постоянного физического копирования папок делает структуру проекта громоздкой и затрудняет навигацию. В ходе настройки я столкнулась с тем, что svn крайне чувствителен к путям в файловой системе(наличие пробелов или использование облачных директорий (OneDrive) легко приводит к критическим ошибкам, чего практически не наблюдалось при работе с git). В итоге работа с svn показалась мне излишне тяжеловесной, поэтому я соглашаюсь с превосходством git как более лаконичного и эффективного средства для решения задач программной инженерии.